# NeuraSched: Intelligent Task Scheduler

Project guide: architecture, training, tuned values, file map, likely questions, and team contributions

Every number in this guide was read from the project files: `src/config.py`, `checkpoints/best_d3qn_scheduler.pt`, `checkpoints/tuning_summary.json` and `benchmark_results.json`.


1. What the project does
2. The simulated cluster and workload
3. Model architecture
4. How one scheduling decision is made
5. How the model is trained
6. Best trained parameters and hyperparameters
7. Benchmark results
8. What each file does
9. Questions you are likely to be asked
10. Contribution split (3 students)


## 1. What the project does

A cluster receives a stream of tasks. Each task needs some CPU cores and some RAM, runs for a certain time, has a priority, and has a deadline. The scheduler decides, for every incoming task, **which of 6 machines (nodes) it should run on**, or whether it should wait.

Simple rules such as "first free node" (FIFO) or "least busy node" ignore the fact that nodes are different: some have many CPUs, some have a lot of memory, some are faster. Bad placement wastes resources (for example, a node's CPUs are full while most of its RAM is idle; this is called *resource stranding*) and causes tasks to miss deadlines.

NeuraSched replaces the rule with a neural network, a **Dueling Double Deep Q-Network (D3QN)**. It looks at the task and the current state of all nodes and scores each possible placement. It then picks the best placement that is physically possible. It is compared against five classical schedulers on the same simulated workloads.

## 2. The simulated cluster and workload

### Cluster (`ClusterConfig`)

| Node | Type | CPU cores | RAM (GB) | Speed |
|---|---|---|---|---|
| 0, 1 | General purpose | 64 | 128 | 1.00x |
| 2, 3 | Compute heavy | 96 | 64 | 1.25x (runs tasks 25% faster) |
| 4, 5 | Memory heavy | 32 | 256 | 1.00x |
| **Total** | | **384** | **896** | |

### Workload (`WorkloadConfig`, `TraceGenerator`)

The workload imitates real data-centre traces (Google Borg, Alibaba):

- **Arrivals:** Poisson arrivals at 5.5 tasks/s. With 20% probability a "burst" starts, during which the rate is multiplied by 2.5.
- **Durations:** heavy-tailed Pareto distribution (shape 1.35), clipped to 2–60 s. Most jobs are short and a few are very long.
- **Resource profiles:**
  - 40% balanced;
  - 30% compute-heavy (8–24 cores, 2–16 GB);
  - 30% memory-heavy (1–8 cores, 16–48 GB).
- **Priorities:** 30% Batch, 50% Standard, 20% Latency-Critical.
- **Deadline** = arrival + duration × slack factor. The slack factor is 3.0 for Batch, 1.8 for Standard and 1.2 for Critical, so critical tasks have the least room to wait.

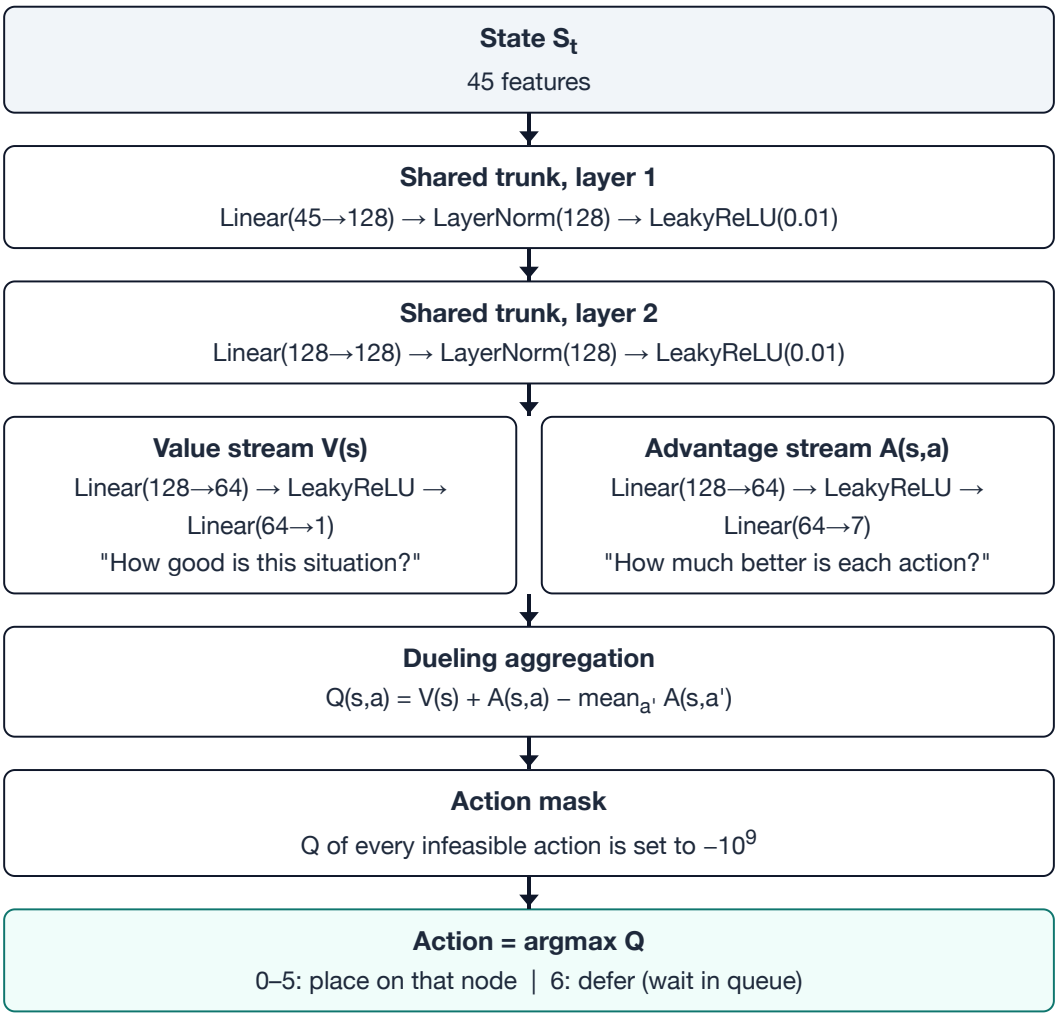
Generation is seeded, so the same seed always produces the same trace. Every scheduler is therefore tested on identical tasks.

## 3. Model architecture

### 3.1 Input: the state vector $S_t$ (45 numbers)

| Part | Size | Contents (all scaled to roughly 0–1) |
|---|---|---|
| Task features | 5 | CPU/32, RAM/64, duration/120, remaining slack before deadline, priority/2 |
| Node features | 6 × 6 = 36 | For each node: free CPU ratio, free RAM ratio, CPU utilization, RAM utilization, free-CPU-to-free-RAM ratio, speed/1.5 |
| Cluster features | 4 | Cluster CPU utilization, cluster RAM utilization, queue backlog/50, load variance across nodes |

### 3.2 Network ( `src/models/dueling_dqn.py` )




- **Total trainable parameters: 39,944** (counted from the saved checkpoint).
- **Weight initialization:** orthogonal, with ReLU gain; biases start at 0.
- **Why LayerNorm:** it keeps the activations on a stable scale, because the input features have different ranges.
- **Why LeakyReLU:** it avoids "dead" neurons that never activate.
- **Why "dueling":**
  - Often several nodes are about equally good, and what matters most is how good the overall state is.
  - Splitting V and A lets the network learn the value of a state without having to learn every action's value separately.
  - Subtracting the mean advantage makes the split unique.
- **Why masking:**
  - A node without enough free CPU *and* RAM can never be chosen, so the model can never overload a machine.
  - "Defer" is only allowed when it makes sense.
  - If no action at all is feasible, the mask is ignored instead of making every value $-10^9$.

## 4. How one scheduling decision is made

1. The simulator advances the clock to the next task's arrival time and frees any tasks that have finished by then.
2. `Cluster.get_state_vector()` builds the 45-number state, and `Cluster.get_action_mask()` marks which of the 7 actions are possible.
3. The network outputs 7 Q-values. Masked actions are pushed to $-10^9$, and the highest remaining one is chosen. This takes about **83 µs** on CPU.
4. If the action is a node, the task is allocated there. Its finish time is start + duration / node speed.
5. If the action is *defer*, the clock jumps to the next task completion. After 2 deferrals the task is forced onto the first feasible node.
6. When every task has been handled, the cluster is drained and the metrics are computed: makespan, turnaround, wait, utilization, stranding and SLA violations.

## 5. How the model is trained

The code contains **two** training methods. It is important to know which one produced the saved model.

> **The saved checkpoint ( `checkpoints/best_d3qn_scheduler.pt` ) was produced by method A, supervised imitation learning (behavioral cloning).** The reinforcement-learning trainer (method B) exists in `src/scheduler/trainer.py` , but the tuning pipeline ( `scripts/tune.py` → `HyperparameterTuner.run_tuning` ) never calls it.

### Method A: Behavioral cloning from an expert (used for the final model, `src/tune/tuner.py` )

1. **Expert ("oracle") policy.** A hand-written rule scores every feasible node:
   `score = −(7 × CPU_util + 3 × RAM_util) + 0.30 (if node is fast) + 0.05 × cosine_alignment`
   In words: prefer the least-loaded node (CPU load counts more than RAM), with a bonus for faster nodes and a small bonus for good resource fit.
2. **Collect demonstrations.** The simulator runs 400-task traces with seeds 1337, 1474, 1611, 100, 101, and so on. At each step it records (state, mask, expert action) until **15,000 samples** are collected.
3. **Supervised training.** The same Dueling network is trained as a classifier: its 7 Q-outputs are treated as logits.
   - Loss: cross-entropy against the expert action.
   - Infeasible actions are masked to $-10^4$ inside the loss.
   - Optimizer: AdamW, learning rate $2\times10^{-3}$.
   - Learning-rate schedule: cosine annealing down to $1\times10^{-5}$.
   - Batch size 64, shuffled; **75 epochs**.
4. **Evaluate and pick the best trial.** Each trial is evaluated on 3 traces of 400 tasks (seeds 1337, 1474, 1611) using a fitness score:
   `fitness = −5·makespan − 4·avg_turnaround − 3·SLA_violation_% + 2·CPU_util_%`
   The best trial's weights are saved to the checkpoint, and the trial history to `tuning_summary.json` . This took about **13 s** on CPU.

### Method B: Deep reinforcement learning, Double DQN with prioritized replay ( `src/scheduler/trainer.py` , implemented but not used for the final model)

- **Environment loop:**
  - Each episode is a new 200-task trace.
  - Actions are chosen ε-greedily, picking only from feasible actions.
  - ε decays linearly from 1.0 to 0.02 over 2,000 steps.
- **Reward** ( `ClusterSimulator.calculate_reward` ): $r = 4.5\cdot r_{sla} + 2.0\cdot r_{pack} + 0.5\cdot r_{balance} + 2.0\cdot r_{speed}$. The four terms are:
  - **$r_{sla}$:** +2 to +4 if the predicted finish time is before the deadline (more for more spare time); otherwise a penalty proportional to lateness, 2× larger for critical tasks.
  - **$r_{pack}$:** cosine similarity between the task's (CPU, RAM) demand and the node's free (CPU, RAM). It rewards good fit and avoids stranding.

- **$r_{balance}$:** $-0.15 \cdot (CPU_util^2 + RAM_util^2)$, which penalizes overloading one node.
- **$r_{speed}$:** +3 for putting a critical task on a fast node (–1.5 otherwise); batch tasks are nudged to slower nodes.
- **Defer:** +0.5 if waiting was really necessary, a large penalty if a node was actually free.

- **Prioritized Experience Replay:**
  - A sum-tree buffer holds 50,000 transitions.
  - Transitions with a large TD error are sampled more often ($\alpha = 0.6$).
  - Importance-sampling weights correct for this bias, with $\beta$ annealed from 0.4 to 1.0.
- **Double DQN target:**
  - The online network picks the best next action (with masking).
  - The target network evaluates it: $y = r + \gamma \cdot Q_{target}(s', \text{argmax}_a\, Q_{online}(s', a))$, with $\gamma = 0.99$.
  - This reduces the over-estimation of normal DQN.
- **Loss and optimizer:**
  - Loss: Huber (Smooth L1), weighted by the importance-sampling weights.
  - Optimizer: AdamW, learning rate $3 \times 10^{-4}$, weight decay $10^{-4}$.
  - Gradient clipping at norm 0.5.
- **Target network:** soft update with $\tau = 0.01$ every 50 steps.
- **Model selection:** evaluated every 10 episodes, and the best weights by (reward – 2·SLA misses – 0.1·makespan) are kept.

## 6. Best trained parameters and hyperparameters

### 6.1 The model that is actually deployed

| Item | Value |
|---|---|
| Winning trial | `NeuraSched_Optimal_D3QN_128` (fitness score –679.6) |
| Training method | Behavioral cloning (supervised) from the expert heuristic |
| Learning rate | **0.002 ($2 \times 10^{-3}$), cosine-annealed to $1 \times 10^{-5}$** |
| Epochs / batch size | 75 / 64 |
| Optimizer | AdamW (default weight decay) |
| Training samples | 15,000 (state, mask, expert action) triples |
| Hidden size / trunk layers | 128 / 2 (+ 64-unit value and advantage heads) |
| Trainable parameters | 39,944 |
| State / action size | 45 / 7 |
| Activation, normalization | LeakyReLU (slope 0.01), LayerNorm |
| Reward weights listed in the trial | packing 2.5, SLA 5.0, balance 0.5, speed 3.0 (stored but not used by behavioral cloning; see section 9) |

### 6.2 RL hyperparameters (`RLConfig`, used by the RL trainer)

| Hyperparameter | Value | Hyperparameter | Value |
|---|---|---|---|
| Learning rate | $3\times10^{-4}$ | Discount γ | 0.99 |
| Batch size | 64 | Replay capacity | 50,000 |
| ε start → min | 1.0 → 0.02 | ε decay steps | 2,000 |
| Target update | every 50 steps, τ = 0.01 | Gradient clip norm | 0.5 |
| PER α | 0.6 | PER β start | 0.4 → 1.0 |
| Hidden dim / layers | 128 / 2 | Device | CPU |

### 6.3 Reward weights (`RewardConfig` defaults)

| $w_{sla}$ | $w_{pack}$ | $w_{balance}$ | $w_{speed}$ | $w_{defer}$ |
|---|---|---|---|---|
| 4.5 | 2.0 | 0.5 | 2.0 | 2.0 |

`scripts/tune.py` also defines a 4-trial search space: learning rates 2.5e-4, 2.0e-4, 3.0e-4, 1.5e-4; hidden sizes 128 and 256; 2 or 3 layers. However, the current `tuning_summary.json` contains only one trial (lr = 0.002), so the saved model came from the tuner's built-in default search space, not from that 4-trial list.

## 7. Benchmark results

Each scheduler ran on 3 seed-locked traces of 400 tasks (1,200 decisions in total). The numbers below are averages from `benchmark_results.json`. **Bold teal** marks the best value in each column.

| Scheduler | Makespan (s) | Avg turnaround (s) | P99 turnaround (s) | Avg wait (s) | CPU util % | RAM util % | Stranded % | SLA viol. % | Critical SLA viol. % | Latency (µs) |
|---|---|---|---|---|---|---|---|---|---|---|
| **D3QN (ours)** | **124.6** | **9.91** | 47.9 | **4.45** | **52.3** | **44.2** | 22.5 | **40.6** | 59.5 | 82.4 |
| FIFO | 135.5 | 11.06 | 52.9 | 5.53 | 48.9 | 41.0 | **20.2** | 44.3 | 61.3 | **0.88** |
| SJF-Backlog | 130.5 | 10.80 | 50.6 | 5.32 | 50.1 | 42.4 | 22.4 | 43.1 | 64.5 | 2.45 |
| EDF | 128.8 | 10.17 | **47.5** | 4.68 | 50.8 | 43.0 | 21.5 | 42.5 | **58.1** | 2.10 |
| Tetris-BestFit | 135.0 | 11.38 | 50.2 | 5.87 | 49.0 | 41.1 | 24.0 | 46.8 | 68.0 | 9.96 |
| Round-Robin | 134.9 | 11.07 | 54.3 | 5.54 | 48.8 | 41.1 | 21.6 | 45.0 | 60.9 | 1.16 |

### How to read it

- **D3QN wins on 6 of the 9 quality metrics (every column except latency):**
  - makespan: 3.3% lower than the best baseline, EDF;
  - average turnaround and average wait;
  - CPU and RAM utilization;
  - overall SLA violation rate: 40.6% vs 42.5% for EDF.
- **It does not win everywhere:**
  - EDF is slightly better on P99 turnaround and on critical-task SLA violations;
  - FIFO has the lowest resource stranding.
- **Decision latency** is about 83 µs, compared with under 10 µs for the heuristics. It is still far below the 1 ms real-time budget, but it is the cost of running a neural network.
- **SLA violation rates are high for every scheduler** (40–47%). The bursty workload overloads the cluster on purpose and critical tasks have very tight deadlines, so the comparison between schedulers is what matters, not the absolute number.

## 8. What each file does

| File | Purpose |
|---|---|
| `src/config.py` | All settings in one place: cluster node specs, workload generator parameters, RL hyperparameters, reward weights. |
| `src/workload/task.py` | The `Task` data class: resources, duration, priority, deadline, plus computed wait time, turnaround and SLA-violated flag. |
| `src/workload/trace_generator.py` | Builds realistic, seeded task streams (bursty Poisson arrivals, Pareto durations, resource profiles, deadlines). CSV save and load. |
| `src/environment/node.py` | One machine: capacity, allocation and release of tasks, utilization, stranding, and its 6-number feature vector. |
| `src/environment/cluster.py` | Manages the 6 nodes. Builds the 45-dim state vector and the 7-action feasibility mask. Computes the cosine packing score. |
| `src/environment/simulator.py` | Discrete-event simulator with a Gym-like `reset()` / `step()`. Handles time, deferral and rewards, and computes all metrics. |
| `src/models/dueling_dqn.py` | The neural network: shared trunk, value and advantage streams, dueling aggregation, action masking, ε-greedy selection. |
| `src/models/replay_buffer.py` | Prioritized Experience Replay using a sum-tree, with importance-sampling weights. |
| `src/scheduler/base.py` | Abstract `BaseScheduler` interface ( `select_node` , `reset` ) shared by every scheduler. |
| `src/scheduler/heuristics.py` | The five baselines: FIFO, SJF-Backlog, EDF, Tetris best-fit, Round-Robin. |
| `src/scheduler/intelligent_scheduler.py` | Loads the trained checkpoint and turns network outputs into node decisions at inference time. |
| `src/scheduler/trainer.py` | The Double-DQN + PER reinforcement learning training loop (method B). |
| `src/tune/tuner.py` | Expert demonstrations, behavioral-cloning training, trial evaluation, fitness scoring, and saving the best checkpoint (method A). |
| `src/benchmarks/benchmark_engine.py` | Runs every scheduler on the same seeded traces, averages the results, prints the table and draws the 4-panel PNG. |
| `scripts/tune.py` | Command line: run hyperparameter tuning and produce the checkpoint. |
| `scripts/run_benchmark.py` | Command line: benchmark D3QN against the baselines; writes `benchmark_results.json` and `benchmark_comparison.png` . |
| `scripts/manual_test.py` | Runs one of the `test_example_*.json` scenarios step by step and prints each decision. |

| | |
|---|---|
| `test_example_1..4.json` | Small hand-made scenarios: priority burst, packing challenge, saturation with pre-loaded nodes, mixed industrial stream. |
| `app.py`, `run_frontend.sh` | Streamlit web interface:<br>• loads the failing / load / passing test cases or your own tasks;<br>• runs the scheduler;<br>• shows per-node CPU/RAM use and tasks allotted, a timeline and the results table;<br>• a Benchmarks tab. |
| `checkpoints/` | Trained weights ( `best_d3qn_scheduler.pt` ) and tuning history ( `tuning_summary.json` ). |
| `tests/` | 10 pytest unit and integration tests for the environment, workload, model and schedulers. All 10 pass. |
| `architecture_spec.md`, `README.md` | Design document and project overview. |

## 9. Questions you are likely to be asked

**Q1. Why reinforcement learning or a neural network instead of a fixed rule?**
Scheduling is a sequence of decisions in which today's placement affects tomorrow's free capacity. Fixed rules look at one factor (order, load, deadline, fit). A learned policy can weigh all 45 features together and adapt to node differences. It also runs in under 1 ms, unlike exact optimizers (MILP), which are far too slow for an online stream.

**Q2. What is a Q-value?**
Q(s,a) estimates the total future reward of taking action a in state s and then acting well afterwards. The scheduler chooses the action with the highest Q among the feasible ones.

**Q3. What does "Double" add to DQN?**
Plain DQN uses the same network to choose and to evaluate the next action, which systematically over-estimates Q. Double DQN chooses with the online network and evaluates with a separate, slowly updated target network.

**Q4. What does "Dueling" add?**
It splits Q into a state value V(s) and per-action advantages A(s,a). When many actions are about equally good, which is common when several nodes are free, the network can still learn how good the state is efficiently.

**Q5. How do you guarantee the model never overloads a node?**

Action masking. Before the argmax, any node without enough free CPU *and* RAM gets Q = $-10^9$, so it can never be chosen. This is a hard guarantee, not something the model has to learn.

**Q6. What is the "defer" action, and why limit it?**
Action 6 holds the task until the next task finishes. It helps when no node fits. The reward punishes deferring while a node is free, and after 2 deferrals the simulator forces a placement so that tasks cannot wait forever.

**Q7. Why prioritized experience replay?**
Transitions with large TD errors are the ones the model predicts worst, so replaying them more often speeds up learning. Importance-sampling weights correct the bias that this non-uniform sampling introduces.

**Q8. Why Huber loss and gradient clipping?**
RL targets can be noisy and large. Huber loss is quadratic for small errors and linear for large ones, so outliers do not blow up the gradients. Clipping at norm 0.5 adds further stability.

**Q9. What is resource stranding, and how does the model address it?**
Stranding happens when one resource on a node is full while another is mostly idle (for example, CPU full and RAM empty), so the idle part cannot be used. The cosine packing term rewards placing a task where its (CPU, RAM) shape matches the node's free (CPU, RAM) shape.

**Q10. How was the final model actually trained?**
By behavioral cloning. 15,000 decisions were recorded from an expert heuristic (least-loaded node, preferring fast nodes), and the dueling network was trained with cross-entropy to copy them: lr 2e-3, cosine schedule, 75 epochs, batch 64. The Double-DQN + PER trainer is implemented and tested, but it was not used to produce the shipped checkpoint. Be ready to say this clearly: imitation learning gave a stable, strong starting policy, and RL fine-tuning from it is the natural next step.

**Q11. If it imitates a heuristic, why is it better than the heuristics?**
The expert it copies is not one of the five baselines. It combines load balancing, a speed preference and packing fit, so it is a stronger rule than any single baseline. The network learns that combined behavior and applies it smoothly across all 45 features.

**Q12. How are the evaluation results kept fair and reproducible?**
Every scheduler runs on exactly the same seeded traces (seeds 1337, 1474, 1611; 400 tasks each), in the same simulator, with the same metrics. The results are averaged over the 3 runs.

**Q13. What are the limitations, and how would you improve the project?**
Be honest about these:

1. **Overlapping seeds.** The expert demonstrations use seeds 1337, 1474 and 1611, which are also the evaluation seeds. The evaluation should be repeated on unseen seeds.
2. **Ignored hyperparameters.** `hidden_dim` and `num_layers` are passed to the network but ignored (128 and 2 are hard-coded), so the 256-unit and 3-layer trials would build identical networks.
3. **Unused reward weights.** The reward weights in the tuning trials are not used by behavioral cloning.
4. **Inconsistent documentation.** The README results table and the reward formula in `architecture_spec.md` do not match the current code and `benchmark_results.json`.
5. **Absolute SLA misses are high**, around 40%, because the workload is very heavy.

Improvements: fine-tune the cloned model with the Double-DQN trainer, evaluate on held-out seeds, make the layer sizes configurable, and add more nodes or real trace data.

**Q14. Why a 45-dimensional state? Would more nodes break it?**
45 = 5 task + 6 nodes × 6 features + 4 cluster features. The size grows with the number of nodes, so a different cluster size needs a retrained model. A graph neural network or attention over nodes would remove that limit, at the cost of slower inference.

**Q15. What do the failing, load and passing cases in the app show?**
**Failing:** one task needs 96 cores and 96 GB, which no node has, so it is rejected; two others block both compute nodes, so a critical task misses its deadline.
**Load:** 8 large tasks fill all three node types, later arrivals queue, and every deadline is still met.
**Passing:** each task matches a node type, with no waiting and every deadline met.

## 10. Contribution split (3 students)

The work is divided into three equal parts that roughly match the project's three layers. Each student owns about 1,000 lines of code plus their share of tests, documentation and the presentation.

### Student 1: Simulation environment and workload Environment

- **Cluster model:** `src/environment/node.py` and `cluster.py`, covering heterogeneous nodes, allocation and release, the 45-dim state encoder and the 7-action feasibility mask.
- **Simulator:** `src/environment/simulator.py`, covering the discrete-event clock, the defer logic, the shaped reward function and all evaluation metrics.
- **Workload:** `src/workload/task.py`, `trace_generator.py` and `src/config.py`, covering Borg/Alibaba-style bursty, heavy-tailed trace generation and SLA deadlines.
- **Tests:** `tests/test_environment.py` and `tests/test_workload.py`.
- **Documentation:** `architecture_spec.md` (problem statement, state and reward design).

### Student 2: Deep learning model and training Model

- **Network:** `src/models/dueling_dqn.py`, covering the dueling architecture, LayerNorm/LeakyReLU trunk, orthogonal initialization and action masking.
- **Replay:** `src/models/replay_buffer.py`, the sum-tree prioritized replay with importance-sampling weights.
- **RL training:** `src/scheduler/trainer.py`, covering the Double DQN loop, Huber loss, soft target updates and the ε schedule.
- **Tuning:** `src/tune/tuner.py` and `scripts/tune.py`, covering expert demonstrations, behavioral cloning, the hyperparameter search and the fitness score.
- **Inference:** `src/scheduler/intelligent_scheduler.py` and producing `checkpoints/`.
- **Tests:** `tests/test_models.py`.

### Student 3: Baselines, benchmarking and application Evaluation & UI

- **Baselines:** `src/scheduler/base.py` and `heuristics.py`, covering FIFO, SJF, EDF, Tetris best-fit and Round-Robin.
- **Benchmarks:** `src/benchmarks/benchmark_engine.py` and `scripts/run_benchmark.py`, covering multi-seed evaluation, the results JSON and comparison plots.
- **Scenarios:** `scripts/manual_test.py` and the `test_example_*.json` scenarios.
- **Web app:** `app.py` and `run_frontend.sh`, the Streamlit interface with test-case loader, node utilization view, timeline and benchmarks tab.
- **Tests:** `tests/test_scheduler.py` (end-to-end).
- **Documentation:** `README.md` (setup, results, project structure).

**Shared by all three:** agreeing on the interfaces between the parts (the state vector, the action mask and `BaseScheduler.select_node`), analysing the results, writing the final report, and presenting. Each student presents their own part and should be able to answer the questions in section 9 that relate to it.